

Kubernetes PROJECT

4rsenal

신재섭(조장)

전병욱

정태환

이은석

목차

1. ETCD 백업, 업그레이드

1-1. ETCD 백업

1-2. Cluster Upgrade

2. 역할 기반 제어

2-1. Service Account, Role, RoleBinding 생성

2-2. Service Account, ClusterRole,
ClusterRoleBinding 생성

3. 노드 관리

3-1. 노드 비우기

3-2. Pod Scheduling

4. 파드 생성

4-1. 환경변수, command, args 적용

4-2. Static Pod 생성

4-3. 로그 환경

4-4. Multi Container Pod 생성

5. 쿠버네티스 컨트롤러와 네트워킹

5-1. Rolling Update & Roll Back

5-2. Cluster IP

5-3. NodePort

5-4. Pod Scale-out

6. NetworkPolicy & Ingress

6-1. Network Policy

6-2. Ingress

6-3. Service and DNS Lookup

7. Resource 관리

7-1. emptyDir Volume

7-2. HostPath Volume

7-3. Persistent Volume

7-4. PVC 를 사용하는 애플리케이션 Pod 운영

1. ETCD 백업, 업그레이드

1-1. ETCD 백업

https://127.0.0.1:2379 에서 실행 중인 etcd 의 snapshot 을 생성하고 snapshot 을 etcd-snapshot.db 에 저장합니다.

그런 다음 다시 스냅샷을 복원합니다.

etcdctl 을 사용하여 서버에 연결하기 위해 다음 TLS 인증서/키가 제공됩니다.

CA certificate: /etc/kubernetes/pki/etcd/ca.crt

Client certificate: /etc/kubernetes/pki/etcd/server.crt

Client key: /etc/kubernetes/pki/etcd/server.key

1) 검색 [etcd backup -> volume snapshot]

<https://kubernetes.io/docs/tasks/administer-cluster/configure-upgrade-etcd/>

2) 루트 전환 및 etcd-client 설치

```
# sudo -i
# apt install etcd-client
```

3) 스냅샷 저장

```
# ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.cr
--cert=/etc/kubernetes/pki/etcd/server.crt
--key=/etc/kubernetes/pki/etcd/server.key \
snapshot save etcd-snapshot.db
```

4) 스냅샷 복원

```
# ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.cr
--cert=/etc/kubernetes/pki/etcd/server.crt
--key=/etc/kubernetes/pki/etcd/server.key \
snapshot restore etcd-snapshot.db
```

5) tree 설치

```
# apt install tree
```

6) etcd 확인

tree default.etcd/member

```
root@master:~# tree default.etcd/member
default.etcd/member
├── snap
│   ├── 000000000000000001-000000000000000001.snap
│   └── db
└── wal
    └── 000000000000000000-000000000000000000.wal

3 directories, 3 files
```

1. ETCD 백업, 업그레이드

1-2. Cluster Upgrade

마스터 노드의 모든 Kubernetes control plane 및 node 구성 요소를 버전 1.34.1 버전으로 업그레이드합니다. master 노드를 업그레이드하기 전에 drain 하고 업그레이드 후에 uncordon 해야 합니다.

"주의사항" 반드시 Master Node 에서 root 권한을 가지고 작업을 실행해야 한다.

1) 검색 [kubeadmin upgrade]

<https://kubernetes.io/docs/tasks/administer-cluster/kubeadm/kubeadm-upgrade/>

2) 버전 확인

```
# kubectl get node
```

```
cka@master:~$ kubectl get node
```

NAME	STATUS	ROLES	AGE	VERSION
master	Ready	control-plane	12d	v1.33.11
worker-1	Ready	<none>	12d	v1.33.11
worker-2	Ready	<none>	12d	v1.33.11

3) 업그레이드 버전 결정

```
# sudo apt update
# sudo apt-cache madison kubeadm
```

4) kubeadmin 업그레이드 호출

```
# sudo apt-mark unhold kubeadm && \
# sudo apt-get update && sudo apt-get install -y kubeadm='1.34.1' && \
# sudo apt-mark hold kubeadm
```

5) 업그레이드 버전 선택 후 실행

```
# kubeadm version
# sudo kubeadm upgrade plan
# sudo kubeadm upgrade apply v1.34.1
```

6) 노드를 예약 불가능으로 표시 후 유지 관리 준비

```
# kubectl drain master --ignore-daemonsets
```

7) kubelet 및 kubectl 업그레이드

```
# sudo apt-mark unhold kubeadm && \  
# sudo apt-get update && sudo apt-get install -y kubelet='1.34.1'  
# kubectl='1.34.1' && \  
# sudo apt-mark hold kubelet kubectl
```

8) kubelet 재시작 후 노드 상태 확인

```
# sudo systemctl daemon-reload  
# sudo systemctl restart kubelet  
# kubectl get node
```

```
cka@master:~$ kubectl get node
```

NAME	STATUS	ROLES	AGE	VERSION
master	Ready,SchedulingDisabled	control-plane	12d	v1.34.1
worker-1	Ready	<none>	12d	v1.34.1
worker-2	Ready	<none>	12d	v1.34.1

9) 노드 차단 해제

```
# kubectl uncordon master
```

10) 각 노드 버전 업데이트 확인

```
# kubectl get node
```

```
cka@master:~$ kubectl get node
```

NAME	STATUS	ROLES	AGE	VERSION
master	Ready	control-plane	12d	v1.34.1
worker-1	Ready	<none>	12d	v1.34.1
worker-2	Ready	<none>	12d	v1.34.1

2. 역할 기반 제어

2-1. Service Account, Role, RoleBinding 생성

애플리케이션 운영 중 특정 namespace 의 Pod 들을 모니터할 수 있는 서비스가 요청되었습니다. api-access 네임스페이스의 모든 pod 를 view 할 수 있도록 다음의 작업을 진행하시오.

1. api-access 라는 새로운 namespace 에 pod-viewer 라는 이름의 Service Account 를 만듭니다.
2. podreader-role 이라는 이름의 Role 과 podreader-rolebinding 이라는 이름의 RoleBinding 을 만듭니다.
3. 앞서 생성한 ServiceAccount 를 API resource Pod 에 대하여 watch, list, get 을 허용하도록 매핑하시오.

1) 네임 스페이스 생성 및 확인 [api-access]

```
# kubectl create ns api-access
# kubectl get ns
```

```
cka@master:~$ kubectl get ns
NAME              STATUS    AGE
api-access        Active    4s
```

2) 서비스 어카운트 생성 및 확인 [pod-viewer]

```
# kubectl create sa pod-viewer -n api-access
# kubectl get sa -n api-access
```

```
cka@master:~$ kubectl get sa -n api-access
NAME          SECRETS  AGE
default       0        3m20s
pod-viewer    0        115s
```

3) 룰 생성 및 확인 [podreader-role]

```
# kubectl create role podreader-role -n api-access --resource=pod --
verb=watch,list,get
# kubectl describe role -n api-access
```

```
cka@master:~$ kubectl describe role -n api-access
Name:          podreader-role
Labels:        <none>
Annotations:   <none>
PolicyRule:
  Resources      Non-Resource URLs  Resource Names  Verbs
  -----
pods            []                 []              [watch list get]
```


4) 롤바인딩 (서비스어카운트 + 롤 결합) [podreader-rolebinding]

```
# kubectl create rolebinding podreader-rolebinding --serviceaccount=api-  
access:pod-viewer --role=podreader-role -n  
# kubectl describe rolebinding -n api-access
```

```
cka@master:~$ kubectl describe rolebinding -n api-access  
Name:          podreader-rolebinding  
Labels:        <none>  
Annotations:   <none>  
Role:  
  Kind:  Role  
  Name:  podreader-role  
Subjects:  
  Kind          Name          Namespace  
  ----          -
```

Kind	Name	Namespace
ServiceAccount	pod-viewer	api-account

2. 역할 기반 제어

2-2. Service Account, ClusterRole, ClusterRoleBinding 생성

애플리케이션 배포를 위해 새로운 ClusterRole 을 생성하고 특정 namespace 의 ServiceAccount 를 바인드하시오.

다음의 resource type 에서만 Create 가 허용된 ClusterRole deployment-clusterrole 을 생성합니다.

Resource Type: Deployment StatefulSet DaemonSet

미리 생성된 namespace api-access 에 cicd-token 이라는 새로운 ServiceAccount 를 만듭니다.

ClusterRole deployment-clusterrole 을 namespace api-access 로 제한된 새 ServiceAccount cicd-token 에 바인딩하세요.

1) 네임 스페이스 확인 [api-access]

```
# kubectl get ns
```

```
cka@master:~$ kubectl get ns
NAME                STATUS  AGE
api-access          Active  4s
```

2) 서비스 어카운트 생성 및 확인 [cicd-token]

```
# kubectl create sa cicd-token -n api-access
```

```
# kubectl get sa -n api-access cicd-token
```

```
cka@master:~$ kubectl describe sa -n api-access cicd-token
Name:                cicd-token
Namespace:           api-access
Labels:              <none>
Annotations:         <none>
Image pull secrets:  <none>
Mountable secrets:   <none>
Tokens:              <none>
Events:              <none>
```

3) 클러스터롤 생성 및 확인 [deployment-clusterrole]

```
# kubectl create clusterrole deployment-clusterrole --
```

```
resource=deployment,statefulset,daemonset --verb=create
```

```
# kubectl describe -n api-access clusterrole deployment-clusterrole
```

```
cka@master:~$ kubectl describe -n api-access clusterrole deployment-clusterrole
Name:                deployment-clusterrole
Labels:              <none>
Annotations:         <none>
PolicyRule:
  Resources            Non-Resource URLs  Resource Names      Verbs
  -----
  daemonsets.apps      []                 []                  [create]
  deployments.apps      []                 []                  [create]
  statefulsets.apps    []                 []                  [create]
```

4) 클러스터를 바인딩 생성 및 확인 [deployment-clusterrolebinding]

```
# kubectl create clusterrolebinding deployment-clusterrolebinding --  
serviceaccount=api-access:cicd-token --clusterrole=deployment-clusterrole -n api-  
access  
# kubectl describe -n api-access clusterrolebinding deployment-clusterrolebinding
```

```
cka@master:~$ kubectl describe -n api-access clusterrolebinding deployment-clusterrolebinding  
Name:          deployment-clusterrolebinding  
Labels:        <none>  
Annotations:   <none>  
Role:  
  Kind: ClusterRole  
  Name: deploymentclusterrole  
Subjects:  
  Kind      Name      Namespace  
  ----      -
```

Kind	Name	Namespace
ServiceAccount	cicd-token	api-access

3. 노드 관리

3-1. 노드 비우기

k8s-worker2 노드를 스케줄링 불가능하게 설정하고, 해당 노드에서 실행 중인 모든 Pod 을 다른 node 로 reschedule 하세요.

1) worker-2 노드 상태 확인

```
# kubectl get node
```

```
cka@master:~$ kubectl get node
```

NAME	STATUS	ROLES	AGE	VERSION
master	Ready	control-plane	12d	v1.33.11
worker-1	Ready	<none>	12d	v1.33.11
worker-2	Ready	<none>	12d	v1.33.11

2) woker-2 노드 드레인 설정

```
# kubectl drain worker-2 --ignore-daemonsets
```

```
cka@master:~$ kubectl drain worker-2 --ignore-daemonsets
node/worker-2 already cordoned
Warning: ignoring DaemonSet-managed Pods: calico-system/calico-node-67ngp, calico-system/csi-node-driver-s7vnm, kube-system/kube-proxy-zvzjq
node/worker-2 drained
```

3) worker-2 노드 드레인 설정 확인

```
# kubectl get node
```

```
cka@master:~$ kubectl get node
```

NAME	STATUS	ROLES	AGE	VERSION
master	Ready	control-plane	12d	v1.33.11
worker-1	Ready	<none>	12d	v1.33.11
worker-2	Ready,SchedulingDisabled	<none>	12d	v1.33.11

4) worker-2 노드 드레인 설정 해제 및 노드 상태 확인

```
# kubectl uncordon worker-2
```

```
cka@master:~$ kubectl uncordon worker-2
node/worker-2 uncordoned
cka@master:~$ kubectl get node
```

NAME	STATUS	ROLES	AGE	VERSION
master	Ready	control-plane	12d	v1.33.11
worker-1	Ready	<none>	12d	v1.33.11
worker-2	Ready	<none>	12d	v1.33.11

3. 노드 관리

3-2. Pod Scheduling

다음의 조건으로 pod 를 생성하세요.

Name: eshop-store

Image: nginx

NodeSelector: disktype=ssd

1) 검색 [nodeselector]

<https://kubernetes.io/docs/tasks/configure-pod-container/assign-pods-nodes/>

2) 노드에 라벨 추가

```
# kubectl label node worker-1 disktype=ssd
```

```
# kubectl label node worker-2 disktype=hdd
```

3) 노드 라벨 확인

```
# kubectl get no -L disktype
```

```
cka@master:~$ kubectl get no -L disktype
```

NAME	STATUS	ROLES	AGE	VERSION	DISKTYPE
master	Ready	control-plane	12d	v1.33.11	
worker-1	Ready	<none>	12d	v1.33.11	ssd
worker-2	Ready	<none>	12d	v1.33.11	hdd

4) 파드용 YAML 파일 생성 [eshop-store.yaml]

```
# kubectl run eshop-store --image=ghcr.io/mjc-t/nginx --dry-run=client -o yaml > eshop-store.yaml
```

5) YAML 파일 수정 [eshop-store.yaml]

```
# vi eshop-store.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    name: eshop-store
spec:
  containers:
    - image: nginx
      name: eshop-store
  nodeSelector:
    disktype: ssd
```

6) YAML 파일 적용 [eshop-store.yaml]

```
# kubectl apply -f eshop-store.yaml
```

7) 파드 생성 확인 [eshop-store]

```
# kubectl get po eshop-store -o wide
```

```
cka@master:~$ kubectl get po eshop-store -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE
READINESS GATES							
eshop-store	1/1	Running	0	39s	10.244.226.69	worker-1	<none>
<none>							

4. 파드 생성

4-1. 환경변수, command, args 적용

'cka-exam'이라는 namespace 를만들고, 'cka-exam' namespace 에 아래와 같은 Pod 를 생성하시오.

pod Name: pod-01

image: busybox

환경변수 : CERT = "CKA-cert"

command: /bin/sh

args: "-c", "while true; do echo \$(CERT); sleep 10;done"

1) 검색 [command]

<https://kubernetes.io/docs/tasks/inject-data-application/define-command-argument-container/>

2) 네임스페이스 생성 및 확인 [cka-exam]

```
# kubectl create ns cka-exam
# kubectl get ns
```

```
cka@master:~$ kubectl create ns cka-exam
namespace/cka-exam created
cka@master:~$ kubectl get ns
NAME                STATUS   AGE
api-access          Active   25m
calico-apiserver    Active   12d
calico-system       Active   12d
cka-exam            Active   6s
default             Active   12d
kube-node-lease     Active   12d
kube-public         Active   12d
kube-system         Active   12d
tigera-operator     Active   12d
```

3) 파드용 YAML 파일 생성 [pod-01.yaml]

```
# kubectl run pod-01 -n cka-exam --image=ghcr.io/mjc-t/busybox --env=CERT="CKA-cert" --dry-run=client -o yaml > pod-01.yaml
```

4) YAML 파일 수정 [pod-01.yaml]

```
# vi pod-01.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-01
  namespace: cka-exam
spec:
  containers:
  - env:
    - name: CERT
      value: CKA-cert
    command: ["/bin/sh"]
    args: ["-c", "while true; do echo ${CERT}; sleep 10;done"]
    image: ghcr.io/mjc-t/busybox
    name: pod-01
```

5) YAML 파일 적용 [pod-01.yaml]

```
# kubectl apply -f pod-01.yaml
```

6) 파드 생성 확인 [pod-01]

```
# Kubectl get po pod-01 -n cka-exam -o wide
```

```
cka@master:~$ kubectl get po pod-01 -n cka-exam -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE          NOMINATED NODE   READINESS GATES
pod-01        1/1     Running   0           19s   10.244.226.70 worker-1      <none>           <none>
```


4. 파드 생성

4-2. Static Pod 생성

worker1 노드에 nginx-static-pod.yaml 라는 이름의 Static Pod 를 생성하세요.

pod name: nginx-static-pod

image: nginx

port : 80

1) YAML 파일 생성 [nginx-static-pod.yaml]

```
# kubectl run nginx-static-pod --image=ghcr.io/mjc-t/nginx --port 80 --dry-run=client -o yaml > nginx-static-pod.yaml
```

2) YAML 파일 수정 [nginx-static-pod.yaml]

```
# vi nginx-static-pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-static-pod
spec:
  containers:
  - image: ghcr.io/mjc-t/nginx
    name: nginx-static-pod
    ports:
    - containerPort: 80
```

3) worker-1 에서 root 로 전환

```
# sudo -i
```

4) static pod 경로로 이동

```
# cd /etc/kubernetes/manifests
```

5) worker-1 YAML 파일 생성 및 수정 [nginx-static-pod.yaml]

vi nginx-static-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-static-pod
spec:
  containers:
  - image: ghcr.io/mjc-t/nginx
    name: nginx-static-pod
    ports:
    - containerPort: 80
```

6) 스택 파드 생성 확인 [nginx-static-pod-worker-1]

kubectl get pod nginx-static-pod-worker-1 -o wide

```
cka@master:~$ kubectl get pod nginx-static-pod-worker-1 -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP             NODE       NOMINATED NODE   READINESS GATES
nginx-static-pod-worker-1          1/1     Running   0           51s   10.244.226.71  worker-1   <none>           <none>
```

4. 파드 생성

4-3. 로그 환경

Pod "nginx-static-pod-k8s-worker-1"의 log 를 모니터링하고, 메시지를 포함하는 로그라인을 추출하세요.
추출된 결과는 /home/ubuntu/pod-log 에 기록하세요.

1) 파드 확인 [nginx-static-pod-worker-1]

```
# kubectl get pod nginx-static-pod-worker-1 -o wide
```

```
cka@master:~$ kubectl get pod nginx-static-pod-worker-1 -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NO
MINATED	NODE	READINESS	GATES				
nginx-static-pod-worker-1	1/1	Running	0	105s	10.244.226.71	worker-1	<n
one>	<none>						

2) 로그 모니터링 [nginx-static-pod-worker-1]

```
# kubectl logs nginx-static-pod-worker-1
```

```
cka@master:~$ kubectl logs nginx-static-pod-worker-1
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/05/11 09:18:21 [notice] 1#1: using the "epoll" event method
2026/05/11 09:18:21 [notice] 1#1: nginx/1.29.8
2026/05/11 09:18:21 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/05/11 09:18:21 [notice] 1#1: OS: Linux 6.8.0-110-generic
2026/05/11 09:18:21 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/05/11 09:18:21 [notice] 1#1: start worker processes
2026/05/11 09:18:21 [notice] 1#1: start worker process 29
2026/05/11 09:18:21 [notice] 1#1: start worker process 30
```

3) 로그 저장 및 로그 확인 [pod-log]

```
# kubectl logs nginx-static-pod-worker-1 > /home/cka/pod-log
```

```
# cat pod-log
```

```
cka@master:~$ kubectl logs nginx-static-pod-worker-1 > /home/cka/pod-log
cka@master:~$ cat pod-log
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/05/11 09:18:21 [notice] 1#1: using the "epoll" event method
2026/05/11 09:18:21 [notice] 1#1: nginx/1.29.8
2026/05/11 09:18:21 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/05/11 09:18:21 [notice] 1#1: OS: Linux 6.8.0-110-generic
2026/05/11 09:18:21 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/05/11 09:18:21 [notice] 1#1: start worker processes
2026/05/11 09:18:21 [notice] 1#1: start worker process 29
2026/05/11 09:18:21 [notice] 1#1: start worker process 30
```

4. 파드 생성

4-4. Multi Container Pod 생성

4 개의 컨테이너를 동작시키는 eshop-frontend Pod 를 생성하시오.

pod image: nginx, redis, memcached, consul

1) YAML 파일 생성 [eshop-frontend.yaml]

```
# kubectl run eshop-frontend --image=ghcr.io/mjc-t/nginx --dry-run=client -o yaml  
> eshop-frontend.yaml
```

2) YAML 파일 수정 [eshop-frontend.yaml]

```
# vi eshop-frontend.yaml
```

```
apiVersion: v1  
kind: Pod  
metadata:  
  name: eshop-frontend  
spec:  
  containers:  
    - image: nginx  
      name: nginx  
    - image: redis  
      name: redis  
    - image: memcached  
      name: memcached  
    - image: consul  
      name: consul
```

3) YAML 파일 적용 [eshop-frontend.yaml]

```
# kubectl apply -f eshop-frontend.yaml
```

4) 파드 생성 확인 [eshop-frontend]

```
# kubectl get po eshop-frontend
```

```
cka@master:~$ kubectl get po eshop-frontend  
NAME             READY   STATUS             RESTARTS   AGE  
eshop-frontend   0/4     ContainerCreating   0           7s
```

5. 쿠버네티스 컨트롤러와 네트워킹

5-1. Rolling Update & Roll Back

Deployment 를 이용해 nginx 파드를 3 개 배포한 다음 컨테이너 이미지 버전을 rolling update 하고 update record 를 기록합니다.

마지막으로 컨테이너 이미지를 previous version 으로 roll back 합니다.

name: eshop-payment

Image : nginx

Image version: 1.16

update image version: 1.17

label: app=payment, environment=production

1) 디플로이먼트용 YAML 파일 생성 [eshop-payment.yaml]

```
# kubectl create deploy eshop-payment --image=ghcr.io/mjc-t/nginx:1.16 --  
replicas=3 --dry-run=client -o yaml > eshop-payment.yaml
```

2) YAML 파일 수정 (label 추가) [eshop-payment.yaml]

```
# vi eshop-payment.yaml
```

```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  labels:  
    app: payment  
    environment: production  
  name: eshop-payment  
spec:  
  replicas: 3  
  selector:  
    matchLabels:  
      app: payment  
      environment: production  
  template:  
    metadata:  
      labels:  
        app: payment  
        environment: production  
    spec:  
      containers:  
        - image: nginx:1.16  
          name: nginx
```

3) YAML 파일 적용 및 기록 [eshop-payment.yaml]

```
# kubectl apply -f eshop-payment.yaml --record
```

4) 파드 및 이미지 버전 확인 [eshop-payment-7655f59d95-skp65]

```
# kubectl describe pod eshop-payment-7655f59d95-skp65
```

```
cka@master:~$ kubectl describe pod eshop-payment-7655f59d95-skp65
Name:          eshop-payment-7655f59d95-skp65
Namespace:     default
Priority:       0
Service Account: default
Node:          worker-2/192.168.111.30
Start Time:    Mon, 11 May 2026 18:43:20 +0900
Labels:        app=payment
               environment=production
               pod-template-hash=7655f59d95
Annotations:   cni.projectcalico.org/containerID: 516f1781a675dc2
               8f7deaba98532de632cfd0f
               cni.projectcalico.org/podIP: 10.244.133.198/32
               cni.projectcalico.org/podIPs: 10.244.133.198/32
Status:        Running
IP:            10.244.133.198
IPs:
  IP:          10.244.133.198
Controlled By: ReplicaSet/eshop-payment-7655f59d95
Containers:
  nginx:
    Container ID:  containerd://4ec04107baad925b724584cd741cb60bd45
    7461
    Image:          nginx:1.16
```

5) 롤링 업데이트

```
# kubectl set image deploy eshop-payment nginx=ghcr.io/mjc-t/nginx:1.17 --record
```

6) 파드 및 이미지 버전 롤링 업데이트 확인 [eshop-payment-69b9cf5d78-5jpk1]

```
# kubectl describe pod eshop-payment-69b9cf5d78-5jpk1
```

```
cka@master:~$ kubectl describe pod eshop-payment-69b9cf5d78-5jpk1
Name:          eshop-payment-69b9cf5d78-5jpk1
Namespace:     default
Priority:       0
Service Account: default
Node:          worker-1/192.168.111.20
Start Time:    Mon, 11 May 2026 18:41:39 +0900
Labels:        app=payment
               environment=production
               pod-template-hash=69b9cf5d78
Annotations:   cni.projectcalico.org/containerID: 5099970711f6deb487b130d3524194dfbe0613
               cni.projectcalico.org/podIP: 10.244.226.74/32
               cni.projectcalico.org/podIPs: 10.244.226.74/32
Status:        Running
IP:            10.244.226.74
IPs:
  IP:          10.244.226.74
Controlled By: ReplicaSet/eshop-payment-69b9cf5d78
Containers:
  nginx:
    Container ID:   containerd://f39c0152cd592aecbab2d29f697a03eec6b4b68
    Image:          nginx:1.17
```

7) 롤백

```
# kubectl rollout undo deploy eshop-payment
```


5. 쿠버네티스 컨트롤러와 네트워킹

5-2. Cluster IP

devops' namespace 에서 deployment eshop-order 를 다음 조건으로 생성하시오.

- image: nginx, replicas: 2, label: name=order

'eshop-order' deployment 의 Service 를 만드세요.

Service Name: eshop-order-svc

Type: ClusterIP

Port: 80

1) 네임스페이스 생성 [devops]

```
# kubectl create ns devops
```

2) 디플로이먼트용 YAML 파일 생성 [eshop-order.yaml]

```
# kubectl create deploy eshop-order -n devops --image=ghcr.io/mjc-t/nginx --replicas=2 --dry-run=client -o yaml > eshop-order.yaml
```

3) YAML 파일 수정(label 수정) [eshop-order.yaml]

```
# vi eshop-order.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: eshop-order
    name: eshop-order
    namespace: devops
spec:
  replicas: 2
  selector:
    matchLabels:
      name: order
  template:
    metadata:
      labels:
        name: order
    spec:
      containers:
        - image: ghcr.io/mjc-t/nginx
          name: nginx
```

4) YAML 파일 적용 [eshop-order.yaml]

```
# kubectl apply -f eshop-order.yaml
```

5) 서비스 생성 [eshop-order-svc]

```
# kubectl expose deploy eshop-order -n devops --name=eshop-order-svc --port=80 --target-port=80
```

6) YAML 파일 적용 및 서비스 확인 [eshop-order / eshop-order-svc]

```
# kubectl get deploy eshop-order -n devops
# kubectl get svc eshop-order-svc -n devops
```

```
cka@master:~$ kubectl get deploy eshop-order -n devops
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
eshop-order   2/2     2            2           91s
cka@master:~$ kubectl get svc eshop-order-svc -n devops
NAME          TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
eshop-order-svc ClusterIP     10.110.94.25 <none>        80/TCP     3m1s
```

5. 쿠버네티스 컨트롤러와 네트워킹

5-3. NodePort

'front-end' deployment 를 다음 조건으로 생성하시오.

image: nginx, replicas: 2, label: run=nginx

'front-end' deployment 의 nginx 컨테이너를 expose 하는 'front-end-nodesvc'라는 새 service 를 만듭니다.

Front-end 로 동작중인 Pod 에는 node 의 30200 포트로 접속되어야 합니다

1) 검색 [NodePort]

<https://kubernetes.io/docs/concepts/services-networking/service/#type-nodeport>

2) 디플로이먼트용 YAML 파일 생성 [front-end.yaml]

```
# kubectl create deploy front-end --image=ghcr.io/mjc-t/nginx --replicas=2 --dry-run=client -o yaml > front-end.yaml
```

3) YAML 파일 수정 (label 수정) [front-end.yaml]

```
# vi front-end.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    run: nginx
  name: front-end
spec:
  replicas: 2
  selector:
    matchLabels:
      run: nginx
  template:
    metadata:
      labels:
        run: nginx
    spec:
      containers:
        - image: ghcr.io/mjc-t/nginx
          name: nginx
```

4) YAML 파일 적용 [front-end.yaml]

```
# kubectl apply -f front-end.yaml
```

5) 서비스용 YAML 파일 생성 [front-end-nodesvc.yaml]

```
# kubectl expose deploy front-end --name=front-end-nodesvc --port=80 --target-port=80 --type=NodePort --dry-run=client -o yaml > front-end-nodesvc.yaml
```

6) 서비스용 YAML 파일 수정 [front-end-nodesvc.yaml]

```
# vi front-end-nodesvc.yaml
```

```
apiVersion: v1
kind: Service
metadata:
  labels:
    run: nginx
  name: front-end-nodesvc
spec:
  ports:
    - port: 80
      protocol: TCP
      targetPort: 80
      nodePort: 30200
  selector:
    run: nginx
  type: NodePort
```

7) 서비스용 YAML 파일 적용 [front-end-nodesvc.yaml]

```
# kubectl apply -f front-end-nodesvc.yaml
```

8) 서비스 확인 [front-end-nodesvc]

```
# kubectl get svc
```

```
cka@master:~$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
front-end-nodesvc	NodePort	10.110.248.63	<none>	80:30200/TCP	4s
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	12d

9) 노드 포드 동작 확인

```
# curl 192.168.111.20:30200
```

```
cka@master:~$ curl 192.168.111.20:30200
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

5. 쿠버네티스 컨트롤러와 네트워킹

5-4. Pod Scale-out

실행되고 있는 eshop-order 디플로이먼트의 파드 숫자를 5 개로 늘리시오.

- namespace: devops
- deployment: eshop-order
- replicas: 5

1) 디플로이먼트 확인 [eshop-order]

```
# kubectl get deploy -n devops
```

```
cka@master:~$ kubectl get deploy -n devops
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
eshop-order   2/2     2            2           12m
```

2) 레플리카 늘린 후 확인

```
# kubectl scale deploy eshop-order -n devops --replicas=5
# kubectl get deploy -n devops
```

```
cka@master:~$ kubectl get deploy -n devops
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
eshop-order   5/5     5            5           13m
```

6. NetworkPolicy & Ingress

6-1. Network Policy

customera, customerb 를 생성한 후, 각각 PARTITION=customera, PARTITION=customerb 를 라벨링하시오.

default namespace 에 다음과 같은 pod 를 생성하세요.

name: poc

image: nginx

port: 80

label: app=poc

"partition=customera"를 사용하는 namespace 에서만 poc 의 80 포트로 연결할 수 있도록 default namespace 에 'allow-web-from-customera'라는 network Policy 를 설정하세요.

보안 정책상 다른 namespace 의 접근은 제한합니다.

1) 검색 [netpolicy]

<https://kubernetes.io/docs/concepts/services-networking/network-policies/>

2) 네임스페이스 생성 및 확인 [customera / customerb]

```
# kubectl create ns customera
```

```
# kubectl create ns customerb
```

```
# kubectl get ns customera customerb
```

```
cka@master:~$ kubectl get ns customera customerb
NAME          STATUS   AGE
customera     Active   14s
customerb     Active   8s
```

3) 네임스페이스 라벨 추가

```
# kubectl label ns customera partition=customera
```

```
# kubectl label ns customerb partition=customerb
```

4) 네임스페이스 라벨 확인

```
# kubectl get ns customera --show-labels
```

```
# kubectl get ns customerb --show-labels
```

```
cka@master:~$ kubectl get ns -L partition
NAME          STATUS   AGE   PARTITION
customera     Active   79s   customera
customerb     Active   73s   customerb
```

5) YAML 파일 생성 후 수정 [netpol.yaml]

vi netpol.yaml

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-web-from-customerera
  namespace: default
spec:
  podSelector:
    matchLabels:
      app: poc
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          project: customerera
    - podSelector:
        matchLabels:
          role: frontend
  ports:
  - protocol: TCP
    port: 80
```

6) YAML 파일 적용 [netpol.yaml]

kubectl apply -f netpol.yaml

7) YAML 파일 적용 확인 [allow-web-from-customerera]

kubectl get netpol

```
cka@master:~$ kubectl get netpol
NAME                                POD-SELECTOR  AGE
allow-web-from-customerera         app=poc       9s
```


6. NetworkPolicy & Ingress

6-2. Ingress

Create a new nginx Ingress resource as follows:

- Name: ping
- Namespace: ing-internal
- Exposing service hi on path /hi using service port 5678

1) 검색 [ingress]

<https://kubernetes.io/docs/concepts/services-networking/ingress/>

2) 네임스페이스 생성 [ing-internal]

kubectl create ns ing-internal

```
cka@master:~$ kubectl get ns ing-internal
NAME              STATUS   AGE
ing-internal      Active   9s
```

3) Ingress 용 YAML 생성 및 수정 [ing.yaml]

vi ing.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ping
  namespace: ing-internal
spec:
  ingressClassName: nginx-example
  rules:
  - http:
      paths:
      - path: /hi
        pathType: Prefix
        backend:
          service:
            name: hi
            port:
              number: 5678
```

4) YAML 적용 [ing.yaml]

kubectl apply -f ing.yaml

5) YAMI 적용 확인

```
# kubectl get ing -n ing-internal
```

```
cka@master:~$ kubectl get ing -n ing-internal
```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
ping	nginx-example	*		80	10s

6. NetworkPolicy & Ingress

6-3. Service and DNS Lookup

image nginx 를 사용하는 resolver pod 를 생성하고 resolver-service 라는 service 를 구성합니다.

클러스터 내에서 service 와 pod 이름을 조회할 수 있는지 테스트합니다.

- dns 조회에 사용하는 pod 이미지는 busybox:1.28 이고, service 와 pod 이름 조회는 nslookup 을 사용합니다.

- service 조회 결과는 /home/ubuntu/nginx.svc 에 pod name 조회 결과는 /home/ubuntu/nginx.pod 파일에 기록합니다.

1) 파드 생성 및 확인 [resolver]

```
# kubectl run resolver --image=ghcr.io/mjc-t/nginx --port=80
```

```
# kubectl get pod resolver
```

```
cka@master:~$ kubectl get pod resolver
```

NAME	READY	STATUS	RESTARTS	AGE
resolver	1/1	Running	0	6s

2) 서비스 생성 및 확인 [resolver-service]

```
# kubectl expose pod resolver --name=resolver-service --port=80
```

```
# kubectl get svc resolver-service
```

```
cka@master:~$ kubectl get svc resolver-service
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
resolver-service	ClusterIP	10.102.10.144	<none>	80/TCP	7s

3) DNS 조회(서비스)

```
# kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox --restart=Never --rm -- nslookup 10.102.10.144
```

```
cka@master:~$ kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox --restart=Never --rm -- nslookup 10.102.10.144
```

```
Server:      10.96.0.10
Address:     10.96.0.10:53

144.10.102.10.in-addr.arpa      name = resolver-service.default.svc.cluster.local

pod "test-nslookup" deleted
```

4) DNS 조회(파트) [resolver]

```
# kubectl get pod resolver -o wide
```

```
cka@master:~$ kubectl get pod resolver -o wide
NAME          READY   STATUS    RESTARTS   AGE      IP             NODE       NOMINATED NODE   R
EADINESS GATES
resolver      1/1     Running   0           2m33s    10.244.226.85  worker-1   <none>           <
```

```
# kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox --restart=Never --rm
-- nslookup 10-244-226-85.default.pod.cluster.local
```

```
cka@master:~$ kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox --restart=Never --rm
m -- nslookup 10-244-226-85.default.pod.cluster.local
Server:      10.96.0.10
Address:     10.96.0.10:53

Name:  10-244-226-85.default.pod.cluster.local
Address: 10.244.226.85

pod "test-nslookup" deleted
```

5) 파일 저장 [nginx.svc / nginx.pod]

```
# kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox --restart=Never --rm
-- nslookup 10.102.10.144 > nginx.svc
# kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox --restart=Never --rm
-- nslookup 10-244-226-85.default.pod.cluster.local > nginx.pod
```

```
cka@master:~$ kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox --restart=Never --rm
m -- nslookup 10.102.10.144 > nginx.svc
cka@master:~$ kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox --restart=Never --rm
m -- nslookup 10-244-226-85.default.pod.cluster.local > nginx.pod
```

6) 파일 저장 확인 [nginx.svc / nginx.pod]

cat nginx.svc

```
cka@master:~$ cat nginx.svc
Server:          10.96.0.10
Address:         10.96.0.10:53

144.10.102.10.in-addr.arpa      name = resolver-service.default.svc.cluster.local

pod "test-nslookup" deleted
```

cat nginx.pod

```
cka@master:~$ cat nginx.pod
Server:          10.96.0.10
Address:         10.96.0.10:53

Name:    10-244-226-85.default.pod.cluster.local
Address: 10.244.226.85

pod "test-nslookup" deleted
```

7. Resource 관리

7-1. emptyDir Volume

다음 조건에 맞춰서 nginx 웹서버 pod 가 생성한 로그파일을 받아서 STDOUT 으로 출력하는 busybox 컨테이너를 운영하시오.

Pod Name: weblog

Web container:

- Image: nginx:1.17
- Volume mount : /var/log/nginx
- Readwrite

Log container:

- Image: busybox
- args: /bin/sh, -c, "tail -n+1 -f /data/access.log"
- Volume mount : /data
- readonly

emptyDir 볼륨을 통한 데이터 공유

1) 검색 [emptyDir]

<https://kubernetes.io/docs/concepts/storage/volumes/>

2) 파드용 YAML 파일 생성 [weblog.yaml]

```
# kubectl run weblog --image=ghcr.io/mjc-t/nginx:1.17 --dry-run=client -o yaml > weblog.yaml
```

3) YAML 파일 수정 [weblog.yaml]

vi weblog.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: weblog
spec:
  containers:
    - image: ghcr.io/mjc-t/nginx:1.17
      name: weblog
      volumeMounts:
        - mountPath: /var/log/nginx
          name: weblog
    - image: ghcr.io/mjc-t/busybox
      name: log
      args: [/bin/sh, -c, "tail -n+1 -f /data/access.log"]
      volumeMounts:
        - mountPath: /data
          name: weblog
          readOnly: true
  volumes:
    - name: weblog
      emptyDir: {}
```

4) YAML 파일 적용 및 확인 [weblog.yaml]

kubectl apply -f weblog.yaml

kubectl get pod weblog

cka@master:~\$ kubectl get pod weblog

NAME	READY	STATUS	RESTARTS	AGE
weblog	2/2	Running	0	24s

```
Containers:
  weblog:
    Container ID:   containerd://0fd06e8f954a5dba22244ce6020fadff55d483982ecdf4e0f498fef4b5e100c5
    Image:          ghcr.io/mjc-t/nginx:1.17
    Image ID:       docker.io/library/nginx@sha256:6ffff55753e3b34e36e24e37039ee9eae1fe38a6420d8ae16ef37c92d1eb26699
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Mon, 11 May 2026 20:02:49 +0900
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/log/nginx from weblog (rw)
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-hxh2r (ro)
  log:
    Container ID:   containerd://4f743e1b91b16598ca428ab3dbf8c6fc94774a684bab52c0b0c828ca3c6108b0
    Image:          ghcr.io/mjc-t/busybox
    Image ID:       ghcr.io/mjc-t/busybox@sha256:b8d1827e38a1d49cd17217efd7b07d689e4ea1744e39c7dcbb95533d175bea65
    Port:           <none>
    Host Port:      <none>
    Args:
      /bin/sh
      -c
      tail -n+1 -f /data/access.log
    State:          Running
      Started:      Mon, 11 May 2026 20:02:54 +0900
    Ready:          True
```

7. Resource 관리

7-2. HostPath Volume

1. /data/cka/fluentsd.yaml 파일을 만들어 새로운 Pod 생성하세요.
신규생성 Pod Name: fluentd, image: fluentd, namespace: default)
2. 위 조건을 참고하여 다음 조건에 맞게 볼륨마운트를 설정하세요.
 1. Worker node 의 도커 컨테이너 디렉토리 : /var/lib/docker/containers 동일 디렉토리로 pod 에 마운트 하시오.
 2. Worker node 의 /var/log 디렉토리를 fluentd Pod 에 동일이름의 디렉토리 마운트하시오.

1) 검색 [hostPath]

<https://kubernetes.io/docs/concepts/storage/volumes/>

2) 파드용 YAML 생성 [fluentsd.yaml]

```
# kubectl run fluentd --image=ghcr.io/mjc-t/fluentd --port=80 --dry-run=client -o  
yaml > fluentd.yaml
```

3) YAML 수정 [fluentsd.yaml]

```
# vi fluentd.yaml
```

```
apiVersion: v1  
kind: Pod  
metadata:  
  name: fluentd  
spec:  
  containers:  
  - image: ghcr.io/mjc-t/fluentd  
    name: fluentd  
    ports:  
    - containerPort: 80  
    volumeMounts:  
    - mountPath: /var/lib/docker/containers  
      name: containersdir  
    - mountPath: /var/log  
      name: logdir  
  volumes:  
  - name: containersdir  
    hostPath:  
      path: /var/lib/docker/containers  
  - name: logdir  
    hostPath:  
      path: /var/log
```


4) YAML 파일 적용 및 확인 [fluentd.yaml]

```
# kubectl apply -f fluentd.yaml  
# kubectl describe pod fluentd
```

```
Volumes:  
  containersdir:  
    Type:          HostPath (bare host directory volume)  
    Path:          /var/lib/docker/containers  
    HostPathType:  
  logdir:  
    Type:          HostPath (bare host directory volume)  
    Path:          /var/log  
    HostPathType:
```

7. Resource 관리

7-3. Persistent Volume

pv001 라는 이름으로 size 1Gi, access mode ReadWriteMany 를 사용하여 persistent volume 을 생성합니다.

volume type 은 hostPath 이고 위치는 /tmp/app-config 입니다.

1) 검색 [pv -> pv00 / hostPath]

<https://kubernetes.io/docs/concepts/storage/persistent-volumes/>

2) pv 용 YAML 파일 생성 [pv001.yaml]

vi pv001.yaml

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv001
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteMany
  hostPath:
    path: /tmp/app-config
```

3) YAML 파일 적용 [fluentd.yaml]

kubectl apply -f fluentd.yaml

4) YAML 파일 적용 확인

```
# kubectl describe pv pv001
```

```
cka@master:~$ kubectl describe pv pv001
Name:          pv001
Labels:        <none>
Annotations:   <none>
Finalizers:    [kubernetes.io/pv-protection]
StorageClass:
Status:        Available
Claim:
Reclaim Policy: Retain
Access Modes:  RWX
VolumeMode:    Filesystem
Capacity:      1Gi
Node Affinity: <none>
Message:
Source:
  Type:        HostPath (bare host directory volume)
  Path:        /tmp/app-config
  HostPathType:
Events:        <none>
```

7. Resource 관리

7-4. PVC 를 사용하는 애플리케이션 Pod 운영

다음의 조건에 맞는 새로운 PersistentVolumeClaim 생성하시오.

Name: pv-volume

Class: app-hostpath-sc

Capacity: 10Mi

앞서 생성한 pv-volume PersistentVolumeClaim 을 mount 하는 Pod 를 생성하시오. Name: web-server-pod

Image: nginx

Mount path: /usr/share/nginx/html

Volume 에서 ReadWriteMany 액세스 권한을 가지도록 구성합니다.

1) 검색 [pv -> kind: persistent]

<https://kubernetes.io/docs/concepts/storage/persistent-volumes/>

2) pvc 용 YAML 생성 [pv.yaml]

vi pv.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pv-volume
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 10Mi
```

3) pvc 용 YAML 적용 [pv.yaml]

kubectl apply -f pv.yaml

```
cka@master:~$ kubectl get pvc
NAME          STATUS  VOLUME  CAPACITY  ACCESS MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS
pv-volume     Bound   pv001   1Gi       RWX            <unset>
```

4) 검색 [pv -> kind: pod]

<https://kubernetes.io/docs/concepts/storage/persistent-volumes/>

5) 파드용 YAML 생성 [web-server-pod.yaml]

vi web-server-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: web-server-pod
spec:
  containers:
    - name: myfrontend
      image: nginx
      volumeMounts:
        - mountPath: /usr/share/nginx/html
          name: mypd
  volumes:
    - name: mypd
      persistentVolumeClaim:
        claimName: pv-volume
```

6) 파드용 YAML 적용 [web-server-pod.yaml]

kubectl apply -f web-server-pod.yaml

7) 파드용 YAML 적용 확인 [web-server-pod]

kubectl get pod web-server-pod

```
cka@master:~$ kubectl get pod web-server-pod
NAME                READY   STATUS    RESTARTS   AGE
web-server-pod      1/1     Running   0           6s
```